

HASSAN ALI

23-NTU-CS-1255

BS-AI(5TH)

Question-1 ESP32 Webserver (webserver.cpp)

Part A: Short Questions

1. What is the purpose of `WebServer server(80);` and what does port 80 represent?

The `WebServer server(80)` creates an 'object' that tells the ESP32 to act as a webserver.

Port 80 is the standard doorway for the internet. When you type a website address into a browser without specifying a number, the browser automatically tries to connect through port 80.

2. Explain the role of `server.on("/", handleRoot);` in this program.

This is an instruction for the esp32, If a user visits the main home page represented by `/`, run the function called `handleRoot`.

3. Why is `server.handleClient();` placed inside the `loop()` function? What will happen if it is removed?

This function must stay in the `loop()` because it tells the ESP32 to constantly check if anyone is trying to visit the site.

If removed: The ESP32 will stop listening to the web. Even if it is connected to Wi-Fi, the website will never load because the server isn't actively answering requests.

**4. In `handleRoot()`, explain the statement:
`server.send(200, "text/html", html);`**

This is the server's final reply to a browser.

200: The standard "OK" signal.

"text/html": Tells the browser to interpret the incoming data as a webpage.

html: The actual code/text that makes up the page the user sees.

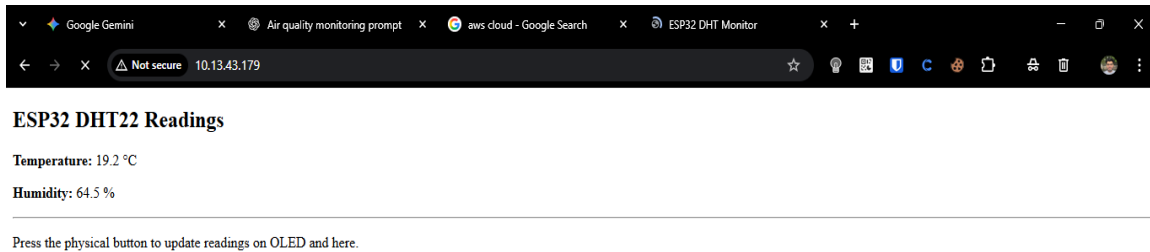
5. What is the difference between displaying last measured sensor values and taking a fresh DHT reading inside `handleRoot()`?

Last Measured: Shows data already stored in memory; it's fast but could be a few seconds old.

Fresh Reading: The ESP32 asks the sensor for new data the moment the page is requested. It's more accurate but can make the webpage feel slightly slower to load.

Part B: Long Question

Describe the complete working of the ESP32 webserver-based temperature and humidity monitoring system.



Connection & IP: The ESP32 connects to local Wi-Fi and is assigned a unique IP address by router. By entering this IP into a phone or computer to access the sensor data.

Request Handling: The server listens for requests; when we visit the page, it triggers a function to gather data and build a response.

Sensor & OLED: The system can read a dht sensor to get climate data. Often, a physical button is used to manually trigger a sensor refresh or update a local oled screen.

Dynamic HTML: The ESP32 doesn't just send a static file; it builds the HTML "on the fly," inserting the live temperature and humidity numbers into the code before sending it.

Meta Refresh: This is a small line of code in the webpage that tells browser to automatically refresh every few seconds, keeping the data on your screen updated without you clicking anything.

Common Issues: Problems often include incorrect Wi-Fi credentials, the ESP32 and phone being on different networks, or the DHT sensor giving NaN (not a number) errors due to loose wiring.

<https://github.com/Rival5555/Embedded-IoT-Systems.git>

Question-2 Blynk Cloud Interfacing (blynk.cpp)

Part-A: Short Questions

1. What is the role of Blynk Template ID in an ESP32 IoT project? Why must it match the cloud template?

This is a unique ID card for your project that tells the Blynk Cloud exactly which dashboard design and settings to use for your hardware.

Matching: If the code and cloud IDs don't match, the Blynk server won't know where to send the data.

2. Differentiate between Blynk Template ID and Blynk Auth Token.

Template ID: defines the *blueprint* or type of project.

Auth Token: Acts as a *specific password* for your individual hardware device to log in to that project.

3. Why does using DHT22 code with a DHT11 sensor produce incorrect readings? Mention one key difference between the two sensors.

The DHT22 is more accurate and has a wider range than the DHT11. If the code is set for one but use the other, the math used to calculate temperature will be wrong, leading to incorrect readings.

4. What are Virtual Pins in Blynk? Why are they preferred over physical GPIO pins for cloud communication?

These are software channels (like V0, V1) used to send data to the app.

Preference: They are better than physical pins because they allow to send complex data (like text or specific colors) without worrying about physical wiring conflicts.

5. What is the purpose of using BlynkTimer instead of delay() in ESP32 IoT applications?

delay() stops the whole esp32, which can cause it to disconnect from the Blynk server.

BlynkTimer allows the esp32 to keep working and stay connected while only performing tasks (like sensor readings) at specific intervals.

Part-B: Long Question

Explain the complete workflow of interfacing ESP32 with Blynk Cloud to display temperature and humidity values.

Setup: You start by creating a Template and Datastreams on the Blynk web dashboard.

Credentials: copy the Template ID, Template Name, and Auth Token into code so the hardware knows where to check in.

Configuration: ensure the DHT library is configured for specific sensor (11 vs 22) to avoid data errors.

Common Problems: Often, users face connection issues because of incorrect Auth Tokens or using delay(), which makes the device appear "Offline" in the app.

<https://github.com/Rival5555/Embedded-IoT-Systems.git>

Screenshots:

1:25

🔔 📶 📶 79% 🔋



DHT11 •

Temperature

Humidity



Air Quality Monitoring x Firebase Pricing x Extensions x Bitwarden Password x Google Gemini x Devices - Blynk.Console x

blynk.cloud/dashboard/945335/global/devices/1/organization/945335/devices/3697606/dashboard

My organization - 1297KH Messages used: 433 of 200.0k

Blynk.Console

- Get Started
- Dashboards
- Custom Data
- Developer Zone
- Devices**
- Automations
- Users
- Organizations
- Locations
- Snapshots
- Fleet Management
- In-App Messaging

DHT11 Online

••••• N7K1 SensorSquad My organization - 1297KH

ⓘ ⚙️ 📄 ⚙️ Edit

Live 1h 6h 1d 1w 1mo 3mo 6mo 1y All

Temperature

20 °C

0 80

Humidity

67.3 %

0 100

Region: SGPL Privacy Policy Terms of Service

